SMART UNIVERSITY MANAGEMENT SYSTEM

SUMS

Course Knowledge Base

Large-Scale Project Programming

SUMS_Course_Knowledge_Base_v1.0

Document Status	Approved Baseline
Version	1.0
Approval Authority	Master Project Control
Scope	Course materials and official project instructions only

This document does not define SUMS requirements, SRS content, database entities, UI screens, or implementation decisions. It becomes an approved baseline only after review and approval in Master Project Control.

Document Control

Field	Value
Document title	SUMS Course Knowledge Base
File name	SUMS_Course_Knowledge_Base_v1.0
Project	Smart University Management System (SUMS)
Course	Large-Scale Project Programming
Version	1.0
Status	Approved Baseline
Owner	SUMS Project Team
Approval location	00 - Master Project Control
Source set	14 uploaded course/project PDF files
Language	English for formal project documentation

Approval Record

Role	Name	Decision	Date	Notes
Project Team Representative	SUMS Project Team	Approved	11-07-2026	Approved Course Knowledge Baseline v1.0
Master Project Control	Master Project Control	Approved	11-07-2026	Final internal approval
Course Instructor / Supervisor (if required)	—	Not requested	—	Internal project baseline

Source Authority and Conflict Resolution

1. Official assignment instructions in final project.pdf have the highest authority for team size, process obligations, originality, submissions, and deadline.
2. Project list.pdf is authoritative for the selected project title and the short official SUMS scope statement.
3. Lecture slides define course concepts, rules, notation, examples, and recommended practices.
4. project example.pdf is an illustrative workflow and expected-shape example; it does not override the official assignment or formal UML rules taught in the lectures.
5. Flowchart exercise files are supporting learning materials and do not by themselves prove that flowcharts are a mandatory submission.
6. When sources conflict, the official assignment takes precedence. The clearest current conflict is team size: Lecture 1 discusses 5-6-person course teams, while final project.pdf explicitly requires 3-4 students; the official 3-4 rule governs SUMS.

Classification Legend

Classification	Meaning
Officially Required	Explicitly stated in final project.pdf or Project list.pdf.
Necessary for the System	Needed to build a coherent, testable, maintainable SUMS even when not listed as a separate submission.
Professional Addition	Improves engineering quality, traceability, or presentation but is not explicitly mandated as a separate deliverable.
Future Enhancement	Deferred capability or advanced practice that must not enter the current scope without approval.
Strongly Expected	Indicated by the official project objectives, course example, or lecture workflow, but not explicitly listed as a separate mandatory submission.

Document Purpose and Boundaries

- This document consolidates definitions, rules, notation, examples, risks, and official obligations from the uploaded course sources.
- It does not create or approve SUMS functional requirements, non-functional requirements, actors, business rules, database entities, classes, UI screens, tests, or code.
- It distinguishes explicit official requirements from necessary engineering work, professional additions, and future enhancements.
- It records ambiguities rather than filling gaps silently.
- Approval of this document authorizes it as a course-reference baseline only; it does not authorize implementation.

Contents

Executive Summary

The uploaded materials define a course approach centered on disciplined software engineering rather than immediate coding. The official project requires a 3-4 student team, SDLC, object-oriented implementation, a relational database, a clear user-friendly interface, original work, professional documentation, and a complete submission package. The lecture set explains how to move from requirements to use cases, prototypes, class and sequence diagrams, implementation, review, quality assurance, and delivery. The Hospital Management System example strongly indicates that requirement analysis, Use Case, Class, Sequence and ER diagrams, prototyping, an MVP/working application, and documentation are expected, but the exact quantity and file structure are not specified in the official assignment.

For SUMS, this knowledge base is a control reference only. It deliberately avoids creating actual SUMS actors, requirements, business rules, classes, tables, screens, or code. Those artifacts must be produced later, reviewed, traced, and approved in their dedicated phases.

Key Conclusions

- Official requirements come from final project.pdf and the SUMS description in Project list.pdf.
- The formal lifecycle must precede and control implementation; code-and-fix is unsuitable.
- Use Case, Class, Sequence, and ER diagrams are strongly expected through the project example and UML objective, but exact counts are not specified.
- The team must use 3-4 students, equal contribution, OOP, a relational database, a user-friendly interface, original work, and a complete submission package.
- Unknown details must remain questions or approved assumptions; they must not become silent requirements.
- The next project action after approving this knowledge base is requirements elicitation and scope analysis, not coding.

Approval status: **Approved by Master Project Control as Course Knowledge Baseline v1.0 on 11-07-2026.**

1. Software Engineering Foundations

Definition from the course materials: Software engineering is the creation and maintenance of software applications by applying technologies and practices from computer science, project management, and other fields. The course emphasizes people working in teams under pressure to create customer value.

Purpose: To transform an idea into a robust deliverable that can evolve, while working with limited time and resources, satisfying the customer, managing risk, and coordinating a team.

Rules and Steps

- Treat software development as a multidisciplinary engineering activity, not only a coding task.
- Balance features, time, and resources; the slides summarize the trade-off as “Good, fast, cheap - choose 2.”
- Plan, execute, test, communicate, and maintain the system through controlled stages.
- Manage expectations and risks early because late defects and late requirement corrections are expensive.
- Use small, frequent source-control updates and break work into smaller estimable tasks.
- Expect plans to change, but require controlled decisions rather than uncontrolled improvisation.

Notation or Format

- Project constraint triangle: Features, Time, and Resources surround the Software Deliverable.
- No dedicated UML notation is attached to the foundation topic.

Examples in the Slides or Files

- Roles of customer/client, managers/designers, developers, testers, and users.
- Failure causes such as unrealistic expectations, shiny-tool optimism, requirement changes, hacks, death marches, “90% done” claims, poor testing response, and weak stakeholder input.
- Mistake categories: People, Process, Product, and Technology.
- Past-student advice: work together, run consistent meetings, estimate smaller chunks, allow time to learn tools, and commit frequently.

Mistakes to Avoid

- Starting with code instead of analysis and planning.
- Treating a new tool or framework as a “silver bullet.”
- Shortchanging upstream analysis, design, or QA under schedule pressure.
- Overpromising progress without measurable evidence.
- Ignoring tester feedback or user input.
- Switching core tools mid-project without a controlled decision.

Effect on SUMS

SUMS must be managed as a real engineering project with baselines, assigned responsibilities, review gates, and measurable progress. The team must not equate “the application runs” with “the project is complete.”

Source / Status	Sources: 01-introduction.pdf, slides 2-13; final project.pdf, pages 1-2. Status: Officially Required + Necessary for the System

2. Software Development Life Cycle

Definition from the course materials: The software lifecycle is the entire process of creating a software product from the initial concept until the last user stops using it.

Purpose: To divide a complex project into reviewable phases, produce tangible outputs, expose errors earlier, and specify what must happen next.

Rules and Steps

- Use identifiable phases: requirements analysis and specification; high-level architectural design; detailed object-oriented design; implementation/integration/debugging; testing/profiling/QA; operation and maintenance.
- Risk assessment and prototyping may be inserted when useful.
- Each phase must have clear activities, a tangible document or item, a review point, and defined inputs for the next phase.
- Do not skip design or testing simply because code can be written quickly.
- Allow feedback and controlled iteration between phases rather than treating a phase as invisible work.
- Find problems as early as possible because the cost of correction grows when discovery is delayed.

Notation or Format

- Typical phase chain: Requirements -> Architecture -> Detailed Design -> Implementation/Integration -> Testing/QA -> Operation/Maintenance.
- Review/verification gates may appear between adjacent phases.

Examples in the Slides or Files

- Lifecycle phases and goals are listed on slide 6.
- The lecture contrasts formal lifecycle management with ad-hoc code-and-fix.
- Project development in stages is also emphasized in Lecture 1.

Mistakes to Avoid

- No formal process or unclear start/finish criteria.
- Producing no reviewable artifact at the end of a phase.
- Beginning implementation before requirements and design are sufficiently approved.
- Treating maintenance and documentation as afterthoughts.
- Using SDLC terminology without actual phase controls.

Effect on SUMS

SUMS should have formal stage gates. Requirements, scope, SRS, use cases, architecture, database, and UI decisions must be approved before full implementation. Testing and documentation must be planned, not postponed to the final days.

Source / Status	Sources: 02-lifecycle.pdf, slides 5-8; 01-introduction.pdf, slide 6; final project.pdf, page 2. Status: Officially Required

3. Lifecycle Models

Definition from the course materials: A lifecycle model organizes how SDLC activities are sequenced, repeated, reviewed, and delivered.

Purpose: To select a workflow that fits requirement stability, risk, schedule certainty, customer feedback needs, and team capability.

Rules and Steps

- Code-and-fix: write, debug, and repeat; it has no formal process and is unsuitable for a large team project.
- Waterfall: execute standard phases in order; use when requirements are stable and formal phase divisions are valuable.
- Spiral: at each loop determine objectives/constraints, identify risks, evaluate options, and develop/verify deliverables.
- Evolutionary prototyping: build a small initial specification and implementation, then evolve both with feedback.
- Staged delivery: define multiple releases early, then design and implement them in sequence; requirements are better known than in evolutionary prototyping.
- Evolutionary delivery: a hybrid; the lecture notes that it can focus on lower-level systems while evolutionary prototyping often focuses on visible front-end aspects.
- Design-to-schedule: prioritize to meet a fixed date; the lecture does not recommend it as a preferred model.
- Agile: adaptive, iterative and incremental work by self-organizing, cross-functional teams.
- Evaluate models using risk management, quality/cost control, predictability, visibility of progress, and customer involvement.

Notation or Format

- Waterfall phase flow with adjacent verification and requirement-change feedback.
- Spiral loops around risk analysis and deliverable verification.
- Staged/evolutionary diagrams show repeated build cycles after initial requirements and architecture.

Examples in the Slides or Files

- Waterfall benefits and drawbacks on slides 9-10.
- Spiral steps and risk-first benefits on slides 11-12.
- Evolutionary prototyping benefits/drawbacks on slides 13-14.
- Staged delivery and evolutionary delivery on slides 15-16.
- Design-to-schedule, design-to-tools, and off-the-shelf options on slide 17.
- Agile values and 12-point manifesto on slides 18-19.

Mistakes to Avoid

- Using code-and-fix for the SUMS project.
- Applying Waterfall so rigidly that approved feedback cannot be incorporated.
- Using Spiral without anyone capable of meaningful risk assessment.
- Letting prototype shortcuts become permanent architecture.
- Using a fixed deadline as an excuse to omit required quality work.
- Claiming to be Agile while having no backlog, feedback, increments, or definition of done.

Effect on SUMS

A controlled hybrid is appropriate for SUMS: approved upstream baselines, early UI prototyping, incremental implementation, frequent integration, and formal final verification. The exact model must be recorded as a project decision rather than assumed.

Source / Status	Sources: 02-lifecycle.pdf, slides 7-21. Status: Necessary for the System + Professional Addition

4. Requirements Engineering

Definition from the course materials: Requirements are goals that the software must accomplish. They describe what the system must achieve, the problem and externally meaningful results, rather than the detailed implementation mechanism.

Purpose: To create an agreed and testable basis for scope, scheduling, design, coding, validation, verification, and acceptance.

Rules and Steps

- Separate “what” from “how.” Requirements should not prematurely prescribe code structure, framework, or exact UI controls.
- Decouple policy from mechanism. Policy states the intended rule or protection; mechanism is the technique used to implement it.
- Iteratively extract and refine requirements; do not expect the first draft to be complete.
- Use requirements as a contractual/expectation basis for customers, a progress basis for managers, a design input for designers, an implementation boundary for coders, and a test basis for QA.
- Use a formal template as a starting point, but do not become a slave to a format that does not fit the project.
- Record unresolved and deferred items explicitly.

Notation or Format

- Cockburn outline: purpose/scope; glossary; use cases; technology; development process; business rules/constraints; performance; security/documentation; usability; portability; unresolved/deferred; human issues.
- Recommended project identifiers (project governance rule, not from the slides): FR-xx, NFR-xx, BR-xx, UC-xx, SEC-xx, ASM-xx, DEC-xx.

Examples in the Slides or Files

- Good policy-level example: an access system protects private features such as employee records.
- Bad over-specific policy example: only the administrator may view employee records.
- Good/bad requirement discussion includes sales tax, lap time display, “never crash,” technology mandates, vague scalability, and a drop-down UI control.
- Y2K is presented as a requirements/abstraction lesson and linked to DRY.

Mistakes to Avoid

- Writing implementation mechanisms as requirements.
- Embedding an exact screen widget in a requirement without justification.
- Writing vague, absolute, or untestable claims such as “never crashes” or “fully secure.”
- Ignoring business rules, constraints, unresolved items, or human issues.
- Mixing requirements with solution design.
- Changing an approved requirement without a change record.

Effect on SUMS

SUMS requirements must be created in a later dedicated phase, assigned stable identifiers, separated by type, reviewed for clarity and testability, and approved before they drive diagrams, database, UI, or code.

Source / Status	Sources: 03-requirements.pdf, slides 3-14; 04-usecases.pdf, slide 2. Status: Officially Required + Necessary for the System

5. Functional and Non-Functional Requirements

Definition from the course materials: Functional requirements map inputs, events, or requests to observable outputs or system behavior. Non-functional requirements define quality attributes, constraints, standards, or operating conditions.

Purpose: To distinguish what the system does from the measurable qualities and constraints under which it must operate.

Rules and Steps

- Functional requirements should state an externally meaningful capability or result.
- Non-functional requirements should be measurable where possible and may cover performance, dependability, security, privacy, safety, reusability, usability, portability, or documentation standards.
- Development quality attributes such as flexibility, maintainability, and reusability are more internal and often more subjective.
- Do not place most non-functional constraints inside a use-case scenario; manage them separately and trace them to verification methods.
- Distinguish non-functional requirements from business rules and implementation constraints.
- Every approved requirement must have a verification approach.

Notation or Format

- Recommended functional pattern: “The system shall <observable function> when <condition>.”
- Recommended non-functional pattern: “The system shall <measurable quality target> under <defined condition>.”
- These patterns are project writing conventions; the slides teach the distinction but do not mandate exact sentence syntax.

Examples in the Slides or Files

- Functional: user can search all databases or a subset; every order receives an ID the user can save.
- Non-functional: deliverable documents conform to a named standard; the system does not disclose personal user information.
- Use-case slide examples of constraints not normally written in use-case steps: database response under two seconds, 99.8% uptime, maximum 200 simultaneous users.

Mistakes to Avoid

- “The system must be fast,” “secure,” “easy,” or “support many users” without metrics.
- Treating a business policy as a non-functional requirement.
- Putting all quality requirements into one vague sentence.
- Failing to trace NFRs to tests or reviews.
- Inventing numerical targets without an official source or approved assumption.

Effect on SUMS

The future SUMS SRS must maintain separate FR and NFR catalogs and must not invent performance, security, availability, or data-volume values silently. Unknown targets must be logged as questions or assumptions for approval.

Source / Status	Sources: 03-requirements.pdf, slides 9-12; 04-usecases.pdf, slide 5. Status: Necessary for the System

6. Requirements Gathering

Definition from the course materials: Requirements gathering is the iterative discovery of user goals, context, constraints, reasons, exceptions, and expectations through interaction and investigation.

Purpose: To discover the real problem and reduce the risk of building a system that matches a document but not the users' needs.

Rules and Steps

- Talk to users or work with them to learn how they actually work.
- Ask questions throughout the process rather than only at the beginning.
- Ask why a user performs a task, not only what steps are currently used.
- Expect requirements to change and manage change explicitly.
- Use use cases and prototypes to expose missing details and exceptions.
- Capture unresolved issues and do not convert guesses into final requirements.
- Avoid collecting unnecessary features merely because they are easy or interesting to implement.

Notation or Format

- Stakeholder interview notes, actor/goal lists, issue logs, assumptions, and prototype feedback records are suitable working formats.
- The slides do not prescribe one elicitation template.

Examples in the Slides or Files

- The "digging" slide lists direct user interaction and continuous questioning.
- Paper prototypes help discover requirements and upcoming design issues.
- The Netflix-style rental exercise is used to discover actors, stakeholders, goals, and formal use cases.

Mistakes to Avoid

- Assuming the team already understands university processes.
- Interviewing only one type of user.
- Recording current manual steps as mandatory future design.
- Trying to think of everything without validation.
- Silently resolving contradictions.
- Treating the short project description as a complete requirement specification.

Effect on SUMS

Before writing SUMS requirements, the team must establish the system boundary, identify stakeholders, list open questions, and secure approval for assumptions. The official description is only a starting point.

Source / Status	Sources: 03-requirements.pdf, slide 6; 04-usecases.pdf, slides 6-15; 06-prototyping.pdf, slides 5-8. Status: Necessary for the System

7. Feature Creep and Scope Control

Definition from the course materials: Feature creep is the gradual accumulation of features over time, often producing a negative overall effect on a large software project.

Purpose: To protect schedule, test capacity, design coherence, and the official project scope from uncontrolled additions.

Rules and Steps

- Establish an approved scope baseline and explicitly list out-of-scope items.
- Require every proposed feature to have a source, purpose, priority, owner, cost, and approval decision.
- Distinguish necessary scope from professional additions and future enhancements.
- Use milestones and priority ordering to protect core deliverables.
- Route changes through Master Project Control and record their effect on requirements, diagrams, database, UI, tests, schedule, and documentation.
- Do not add a feature solely because developers enjoy coding it, users casually mention it, or it improves presentation without supporting a goal.

Notation or Format

- Scope baseline, Change Request, Decision Log, and deferred/future-enhancement list.
- MoSCoW is a reasonable professional prioritization option, but it is not taught in the uploaded slide set and must not be presented as an official course rule.

Examples in the Slides or Files

- The slides explain that developers, marketers, and users may all encourage additional features, while too many features produce more bugs, delays, and less testing.
- “Stone soup” and “boiled frog” analogies illustrate gradual, hard-to-notice growth.
- Lecture 1 lists requirements gold-plating, developer gold-plating, and feature creep as product mistakes.

Mistakes to Avoid

- Adding modules unrelated to the official SUMS description.
- Calling all enhancements “required.”
- Making an approved identifier disappear when scope changes.
- Allowing UI mockups to create unsupported requirements.
- Deferring all testing to make room for extra features.
- Using “future enhancement” as a hidden commitment in the current release.

Effect on SUMS

SUMS must keep a strict baseline. New ideas may be recorded, but they cannot become current requirements until approved. This is essential because the official description is broad and could otherwise grow without limit.

Source / Status	Sources: 03-requirements.pdf, slide 7; 01-introduction.pdf, slide 12. Status: Necessary for the System + Professional Addition

8. Stakeholders and Actors

Definition from the course materials: An actor is anything with behavior that acts on the system. A primary actor initiates interaction to achieve a goal; a supporting actor performs sub-goals. A stakeholder is anyone interested in the system and may not directly act in a scenario.

Purpose: To identify who needs value from the system, who interacts with it, who supplies services, and whose concerns must be represented.

Rules and Steps

- Define the system boundary before deciding whether an entity is an actor.
- Identify primary actors, supporting actors, and stakeholders who do not directly interact.
- Associate actors with goals, not with technical components.
- Classify goals as summary, user, or subfunction where useful.
- Keep distinct roles separate when they have different goals or permissions.
- Do not assume that every stakeholder is an actor or that every internal subsystem is an external actor.

Notation or Format

- Actor/Goal table: ACTOR | USE CASES INITIATED.
- UML summary diagrams may connect stick-figure actors to oval use cases.
- The uploaded use-case slides do not formally teach system-boundary rectangles, <<include>>, or <<extend>> notation; these must not be claimed as slide requirements.

Examples in the Slides or Files

- Club Member, Potential Member, Past Member, Membership Department, Marketing Department, and Services System.
- Netflix-style rental exercise asks students to identify actors and stakeholders.
- The project example uses Admin, Doctor, Nurse, Accountant, and Patient for the Hospital Management System.

Mistakes to Avoid

- Using “User” for all roles when goals differ.
- Treating the database as an actor when it is an internal implementation component.
- Placing an external service inside the system boundary.
- Creating actors from class names or job titles without a goal.
- Assuming SUMS roles before elicitation and approval.

Effect on SUMS

A future SUMS stakeholder register and actor/goal list must be approved before use cases and UI flows are created. This knowledge base does not define the actual actors.

Source / Status	Sources: 04-usecases.pdf, slides 6-9; project example.pdf, pages 1-2. Status: Necessary for the System

9. Use Cases

Definition from the course materials: A use case is a written description of a user's interaction with the software product to accomplish a goal; it captures interactions between an actor and the system.

Purpose: To discover and document functional requirements, define system responsibility, help prioritize work, and expose error cases that must be tested.

Rules and Steps

- Write from the actor's point of view, not from the internal system or developer perspective.
- Focus on a user-visible goal and observable system response.
- Keep the main success scenario clear and typically 3-9 steps long.
- Separate the main success path from extensions and failures.
- Do not use a use case to document most non-functional requirements.
- Use goal-oriented names rather than internal feature names.
- Use cases become inputs to sequence diagrams and system tests.

Notation or Format

- Three styles: actor/goal list, informal paragraph, and formal use case.
- Optional Agile user-story form: "As a [user role], I want to [goal], so I can [reason]."

Examples in the Slides or Files

- Mobile phone: call someone, receive a call, send a message, memorize a number, contrasted with internal functions.
- Netflix-style video rental exercise.
- Informal use case "Customer Loses a Disc."
- Formal use case "Buy Stocks over the Web."
- Agile user story: user wants to log in to access subscriber content.

Mistakes to Avoid

- Writing internal data transmission, database queries, or UI implementation as use-case goals.
- Creating a long procedural document that mixes all paths.
- Embedding response-time or uptime requirements in scenario steps.
- Naming use cases as vague CRUD bundles such as "Manage Everything."
- Creating a use case with no actor goal or success guarantee.

Effect on SUMS

SUMS use cases must later be derived from approved actors and functional requirements. They will provide traceability to sequence diagrams, UI screens, and test cases.

Source / Status	Sources: 04-usecases.pdf, slides 3-16. Status: Necessary for the System + Strongly Expected

10. Use Case Specifications

Definition from the course materials: A formal use-case specification documents the actor goal, preconditions, guarantees, main success scenario, and failure/alternative extensions.

Purpose: To remove ambiguity from a use-case name and provide an agreed behavioral basis for design, UI flows, sequence diagrams, and testing.

Rules and Steps

- Cockburn step 1: identify actors and goals.
- Step 2: write the easiest-to-read main success scenario and capture actor intent and responsibility.
- Step 3: list failure extensions; nearly every step can potentially fail.
- Step 4: describe failure handling as recoverable or non-recoverable.
- Each extension should state the triggering condition and the system's plausible response.
- A response should return to another step or end the use case.
- Do not list failures outside the system's meaningful scope, and do not invent remedies the system cannot implement.

Notation or Format

- Use Case ID and Name; Primary Actor; Level; Preconditions; Minimal Guarantee; Success Guarantee; Main Success Scenario; Extensions.
- Extension numbering such as 1a, 1a1, 2a links alternatives to a main-flow step.
- Goal levels: summary, user, and subfunction.

Examples in the Slides or Files

- "Place an Order" with low-credit and low-stock extensions.
- "Buy Stocks over the Web" includes precondition, minimal guarantee, success guarantees, six main steps, and web-failure extensions.
- Customer Orders a Movie exercise.

Mistakes to Avoid

- No precondition or success guarantee.
- Failure handling that is impossible for the system.
- Assuming the database or network cannot fail.
- Listing "user's power goes out" as a system extension.
- Using vague sentences without naming who performs the action.
- Making every specification far longer than needed.

Effect on SUMS

The SUMS project should eventually create formal specifications for approved, important use cases. The number and depth must be decided in Master Project Control because the official instructions do not specify a count.

Source / Status	Sources: 04-usecases.pdf, slides 10-15. Status: Professional Addition + Necessary for the System

11. UML Use Case Diagrams

Definition from the course materials: The uploaded materials treat a UML use-case summary diagram as a visual actor/goal map showing actors connected to the use cases they initiate or participate in.

Purpose: To provide a quick system-level view of external roles and major user goals.

Rules and Steps

- Use actors to represent external behavioral roles and ovals to represent goal-oriented use cases.
- Connect actors only to use cases in which they participate.
- Use names that describe user goals, not internal modules or database operations.
- Keep the diagram at a summary level; details belong in the written specification.
- Ensure the actor list, functional requirements, and diagram agree.
- Use consistent actor names across SRS, diagrams, UI, and tests.
- The slides do not formally define <<include>>, <<extend>>, actor generalization, or mandatory system-boundary notation; use them only under an approved UML convention and do not overcomplicate the diagram.

Notation or Format

- Stick figure: actor.
- Oval: use case.
- Solid association line: participation/interaction in the course examples.
- A system boundary rectangle is a common professional UML convention, but it is not explicitly taught in the uploaded use-case lecture.

Examples in the Slides or Files

- Actor/goal list and actor-to-use-case summary on slide 8.
- Hospital example connects Admin, Doctor, Nurse, Accountant, and Patient to six use cases.
- The Hospital diagram is illustrative and simplified; it should not override lecture-level UML discipline.

Mistakes to Avoid

- No system scope or inconsistent actor definitions.
- Connecting every actor to every use case.
- Using database tables or code classes as use cases.
- Using implementation details as oval names.
- Excessive include/extend chains without course justification.
- Treating the example's layout as a complete formal UML standard.

Effect on SUMS

SUMS should create one clean top-level use-case diagram after actors and use cases are approved. Additional diagrams may be created only if readability requires decomposition.

Source / Status	Sources: 04-usecases.pdf, slides 6-9; project example.pdf, pages 1-2. Status: Strongly Expected / Necessary for the System

12. UML Class Diagrams

Definition from the course materials: A UML class diagram depicts the static object-oriented structure of the system: classes, attributes, operations, inheritance, associations, multiplicities, aggregation, composition, and dependencies.

Purpose: To translate approved requirements into coherent object responsibilities and relationships before code is written.

Rules and Steps

- Identify candidate classes from nouns in the specification and candidate responsibilities/methods from verbs; validate candidates rather than accepting every noun or verb.
- Use CRC cards to record Class, Responsibilities, and Collaborators.
- Class box sections: name, attributes, and operations. Mark <<interface>> explicitly; italicize abstract class names.
- Attributes should cover object fields and derived properties; trivial getters/setters may be omitted, but interface methods must not be omitted.
- Do not repeat inherited methods in child classes.
- Draw hierarchy top-down with arrows pointing to the parent.
- For associations, specify multiplicity, relationship name, and navigability when useful.
- Protect data, minimize public interfaces, keep high cohesion and low coupling, keep related data and behavior together, avoid model dependence on UI, and eliminate irrelevant classes.

Notation or Format

- Attribute: visibility name : type [count] = defaultValue. Visibility: + public, # protected, - private, ~ package; / derived; underline static.
- Method: visibility name(parameters) : returnType. Parameter form: name : type. Omit return type for constructors and void.
- Multiplicity: * (zero or more), 1 (exactly one), 2..4, 3..*.
- Aggregation: white diamond. Composition: black diamond. Dependency: dotted line/arrow.
- Inheritance styles in the slide: class - solid black arrow; abstract class - solid white arrow; interface - dashed white arrow.

Examples in the Slides or Files

- Student must have exactly one ID card; RectangleList may contain zero or more rectangles.
- Car-Engine aggregation, Book-Page composition, LotteryTicket-Random dependency.
- Texas Hold'em design exercise and class diagram.
- Design heuristics cover Deck API, duplicated Player/Game data, God-class PokerGame, role classes such as Dealer/NextBetter, and policy placement.
- Hospital project example lists User, Patient, Doctor, Appointment, Bill, and InventoryItem, but its diagram is simplified and not a complete notation reference.

Mistakes to Avoid

- God class with most data and behavior.
- Too many tiny classes or classes that are merely roles or operations.
- Duplicating data across classes.
- Exposing an internal collection's entire API.
- Excessive accessors that reveal poor responsibility placement.
- Missing multiplicities or unclear relationships.
- Using foreign-key fields as a substitute for object relationships.
- Mixing ERD notation with class-diagram notation.

Effect on SUMS

SUMS classes must be derived only after approved requirements and use cases. The diagram must remain consistent with sequence messages and OOP implementation. Database tables must not be copied mechanically into classes.

Source / Status	Sources: 07-classdiagrams.pdf, slides 2-29; project example.pdf, pages 3-5. Status: Strongly Expected + Necessary for the System

13. UML Sequence Diagrams

Definition from the course materials: A sequence diagram is an interaction diagram that models one scenario executing in the system and shows messages among participants over time.

Purpose: To transform an approved use-case scenario into object collaboration and verify that responsibilities and method calls are feasible before implementation.

Rules and Steps

- One sequence diagram should model one clear scenario or one manageable branch.
- Participants are arranged horizontally; time advances downward.
- Represent object instances as objectName : ClassName when the name clarifies the diagram.
- Write message names and arguments above call arrows; use dashed arrows for returns.
- Use activation bars while an object's method is active; nest activations for recursion.
- Use opt for optional behavior, alt for alternatives, loop for repetition, and ref for a referenced interaction.
- Creation appears at the creation point and may be marked new/create; destruction is shown with X when relevant.
- Keep the diagram above source-code detail, language-independent, and consistent with class operations and use-case order.

Notation or Format

- Participant/object rectangle; dashed vertical lifeline; activation bar.
- Message arrows: normal/synchronous and asynchronous arrowheads; dashed return arrow.
- Frames: opt [condition], alt [condition] with dashed separator, loop [condition/items], ref interaction name.
- Object creation message and destruction X.

Examples in the Slides or Files

- Facebook user authentication.
- Centralized versus distributed control examples.
- Flawed sequence-diagram exercises.
- Poker Start New Game Round and Betting Round.
- Add Calendar Appointment including validation, conflict, and join-existing-meeting alternatives.
- Hospital Appointment Booking example is a simplified dynamic interaction.

Mistakes to Avoid

- No clear scenario or mixed unrelated use cases.
- Messages in an impossible order.
- Participants that are not actors, objects, or external entities with a real role.
- Method names that do not exist in the class design.
- No guards on alternatives.
- Drawing every line of code.
- Using a generic "System" lifeline for all responsibilities.
- Creating sequence diagrams before use cases and class responsibilities are stable.

Effect on SUMS

SUMS should create sequence diagrams for critical approved scenarios, not for every minor operation. The quantity is currently unspecified and must be approved. Each diagram must trace to a use case and class operations.

Source / Status	Sources: 08-sequencediagrams.pdf, slides 2-22; project example.pdf, page 6. Status: Strongly Expected / Necessary for the System

14. Flowcharts

Definition from the course materials: A flowchart visually presents the flow of data and the sequence of processing operations; it can represent an algorithm and serve as a blueprint for solving a problem.

Purpose: To explain procedural logic, decisions, loops, inputs, outputs, and calculation steps clearly before or alongside implementation.

Rules and Steps

- Begin and end with terminal symbols.
- Use the input/output symbol for reading or displaying data and a process rectangle for calculations/assignments.
- Use a decision diamond for binary conditions and label branches Yes/No or True/False.
- Use flow lines with arrowheads to show direction.
- Initialize variables before use and update loop controls on every repeating path.
- Ensure all paths either continue meaningfully or reach an end.
- Use connectors to avoid confusing line crossings and predefined-process symbols for subroutines.
- Verify loops with a trace table when appropriate.

Notation or Format

- Terminal: rounded rectangle.
- Process: rectangle.
- Input/Output: parallelogram.
- Decision: diamond.
- Connector: circle.
- Predefined Process: rectangle with double vertical edges.
- Flow line: arrow.

Examples in the Slides or Files

- Flowchart Exercises 1: circle area; greatest of two numbers; even numbers 9-100; odd numbers below a limit with sum/count; average of 25 scores; assignments for sum 1-100, largest of three, and prime check.
- Flowchart Exercises 2: feet-to-centimeters; rectangle area; largest of two; power 2^4 directly and with a loop/trace table; add two numbers; negate a number; circle area; positive/negative; quadratic roots; sine 0-360; factorial 20; sum even numbers 0-20; largest in a list; N students/three marks/average/grade; positive and negative totals/counts.

Mistakes to Avoid

- Using a process rectangle for input/output.
- A decision with unlabeled or missing branches.
- A loop with no counter update or termination path.
- Using an uninitialized variable.
- Crossing lines when a connector would be clearer.
- No start or stop.
- Copying an apparent exercise typo without logical validation; page 18 of Flowchart Exercises 2 appears to retain an unrelated output phrase from the previous grading example.

Effect on SUMS

Flowcharts should be used in SUMS only for approved algorithms or complex procedural rules where they add clarity. They are not currently proven to be a mandatory project submission.

Source / Status	Sources: Flowchart Exercises 1 File.pdf, pages 1-5; Flowchart Exercises 2 File.pdf, pages 1-18. Status: Professional Addition

15. User Interface and Prototyping

Definition from the course materials: Usability is the effectiveness with which users achieve tasks in a software environment. Prototyping creates a scaled-down or incomplete system version to demonstrate or test aspects of it.

Purpose: To discover requirements, improve UI design, identify test cases, obtain user feedback, reduce rework, and support team discussion before implementation.

Rules and Steps

- Good UI design focuses on the user rather than the developer or internal system.
- Use user testing/field studies, expert review, card sorting, paper prototyping, or code prototyping as appropriate.
- Perform paper prototyping during or after requirements and before detailed design.
- In a paper-prototype session, give the user tasks; a facilitator guides, one team member can “play computer,” and observers record findings.
- Shneiderman’s Eight Golden Rules: consistency, shortcuts, informative feedback, clear results, simple error handling, easy undo, user control, and reduced short-term memory load.
- Select components according to data and task: buttons for independent actions, checkboxes for independent booleans, radio buttons for one of a few choices, text fields for free input with validation, lists/combos for fixed options, sliders/spinners for bounded numbers.
- Use tabs/pages for frequent switching and use dialogs sparingly for temporary interaction.
- Identify screens from use cases and map navigation before polishing visual style.

Notation or Format

- Paper layers for changing screens; tape for buttons/check boxes; index cards for tabs/dialogs; removable tape for text fields; separate paper for expanded combo boxes; highlighted tape for selections; gray overlays for disabled widgets.

Examples in the Slides or Files

- UI Hall of Shame, bad error messages, Apple/Mac examples, layout and color examples.
- LIBSYS search form component-choice critique.
- Paper prototype background/layer/widget simulations.
- Music player prototyping exercise asking about click count and learnability.

Mistakes to Avoid

- Designing screens before use cases are approved.
- Using generic OK/Yes/No when a clear action verb is available.
- Relying only on tooltips.
- Overusing menus, popups, or wizards.
- Using unlabeled text fields or failing to validate input.
- Treating a prototype as production code.
- Creating attractive screens that do not support an approved actor goal.
- Skipping user task walkthroughs.

Effect on SUMS

SUMS must eventually have a clear, user-friendly interface as an official requirement. Wireframes and prototypes should be tied to approved use cases, reviewed before implementation, and retained as design evidence.

Source / Status	Sources: 06-prototyping.pdf, slides 2-28; final project.pdf, page 2; project example.pdf, page 7. Status: Officially Required + Strongly Expected

16. Team Organization

Definition from the course materials: Team organization assigns roles, responsibilities, communication paths, and work structures so multiple people can deliver one coherent product.

Purpose: To use collective experience while controlling communication problems, diffusion of responsibility, conflict, and unplanned work.

Rules and Steps

- Create a shared mission and goals, clear roles, accountable work ownership, and a results-driven structure.
- Common roles include designers, developers, tech lead, testers, and project manager; in a 3-4 person team, members may hold multiple roles.
- Organize around functionality rather than only job titles.
- Give members specific, small, attainable goals, frequent updates, and small-team collaboration rather than excessive solo work.
- Monitor individual performance and maintain a credible issue/decision communication system.
- Every member must have a significant role and be able to explain the part developed.
- Use kind, specific, quantitative intervention when contribution problems appear.

Notation or Format

- Responsibility Assignment Matrix (e.g., owner/reviewer) is a professional format; the slides do not mandate RACI.
- Team models discussed: business, chief programmer, skunk works, feature, search-and-rescue, SWAT, athletic, theater; dominion and communion structures.

Examples in the Slides or Files

- Benefits of larger teams: bigger problems and collective experience.
- Risks: communication, diffusion of responsibility, inertia, conflict/mistrust.
- Functionality areas: scheduling, development, testing, documentation, release preparation, internal/customer communication.
- Slacker-handling and improved quantitative email example.

Mistakes to Avoid

- One member owns all critical knowledge.
- Responsibilities are implicit or duplicated.
- Members work alone for long periods without review.
- Giving a weak member an isolated critical task without support.
- Accusatory communication.
- Unequal contribution or inability to explain team work.
- Applying the lecture's 5-6 member example instead of the official 3-4 member rule.

Effect on SUMS

SUMS must define owners and reviewers for artifacts and implementation areas. Multiple roles can be combined, but responsibilities and evidence of contribution must be visible.

Source / Status	Sources: 05-teams.pdf, slides 2-17; final project.pdf, pages 1-2; 01-introduction.pdf, slide 6 (superseded team-size example). Status: Officially Required + Necessary for the System

17. Project Management and Decision Making

Definition from the course materials: Project management coordinates decisions, priorities, milestones, communication, responsibilities, meetings, and conflict resolution to move the project toward agreed goals.

Purpose: To make work visible, decisions repeatable, schedules realistic, and disagreements resolvable.

Rules and Steps

- Let all members provide input.
- Compare alternatives using pros/cons, cost/benefit, learning effort, necessity, and risk.
- Strive for consensus; define a fallback such as majority vote and a tie-break rule.
- Possible techniques include stepladder discussion, Pareto analysis, and compromise.
- After a decision, document the decision and responsibilities in a shared location.
- Break the goal into smaller milestones with dates and communication/review points.
- Prioritize tasks by urgency and due date; assign responsible members.
- Run meetings with a pre-shared agenda, progress updates for every work item, decision notes, sketching materials, focused discussion, and a clear action plan.
- Do not meet only to exchange one-way information that could be sent asynchronously.

Notation or Format

- Decision Log, milestone plan, task list/backlog, meeting minutes, and risk log.
- No specific project-management software is mandated.

Examples in the Slides or Files

- Decision methods and Pareto “20% of work solves 80% of the problem.”
- Documenting decisions by email or shared wiki/web page.
- Effective meeting checklist and meeting gotchas.
- Specific, incremental progress and reply dates in team communication.

Mistakes to Avoid

- Unspoken decision authority.
- No documented decision or owner.
- Large tasks with only a final due date.
- Meetings without agenda or action items.
- Ignoring a member’s input.
- Long, unfocused meetings.
- Planning to catch up later or abandoning planning under pressure.
- Changing an approved technical or requirements decision without logging it.

Effect on SUMS

Master Project Control should hold the authoritative Decision, Assumption, Risk, Change, and Progress records for SUMS. Dedicated chats produce artifacts, but they do not independently redefine the baseline.

Source / Status	Sources: 05-teams.pdf, slides 6-8 and 16-19; 01-introduction.pdf, slides 9-12. Status: Professional Addition + Necessary for the System

18. Code Reviews

Definition from the course materials: A code review is a constructive review of another developer's code, often requiring another team member's approval before changes or new code are integrated.

Purpose: To improve maintainability, readability, DRY design, defect detection, shared knowledge, accountability, and developer learning.

Rules and Steps

- Author and reviewer examine a coherent change that is ready for integration and is neither too large nor too small.
- Reviewer comments on logic, structure, and agreed quality standards.
- Feedback leads to refactoring and a second review when needed; approval ends the review.
- More than one person should see every important piece of code.
- Use reviews to make authors explain decisions and to spread system knowledge.
- Inspection is formal with roles and checklists; walkthrough is a less formal author/reviewer discussion; code reading is offline individual review.
- Use a checklist covering coding style, comments, logic, error handling, and coding decisions.
- Keep review constructive and focused on code, not personality.

Notation or Format

- Pull request / review record; reviewer comments; approval/rework state; checklist.
- The slides discuss industrial review tools and SCM integration but do not mandate one product.

Examples in the Slides or Files

- Industry examples from Google, Yelp, Wizards of the Coast, and Facebook.
- Account class review: make fields private, replace magic values, use constants/enum, improve comments, and decompose logic.
- Studies compare defect-detection rates of testing and inspections.

Mistakes to Avoid

- Automatic approval without reading.
- Reviewing huge unrelated changes.
- Reviewing personality or coding ownership rather than the code.
- Using review as punishment.
- Ignoring error handling or design decisions.
- Merging changes no other member understands.
- Treating review as a complete substitute for testing.

Effect on SUMS

When implementation begins, SUMS should use reviewable branches or pull requests, with at least one reviewer for meaningful changes. A lightweight review log supports contribution evidence and shared understanding.

Source / Status	Sources: 09-codereviews.pdf, slides 4-21. Status: Professional Addition

19. Software Quality Assurance

Definition from the course materials: Software QA asks whether the team is building the right system and building it right, and whether the product has the required good properties while avoiding unacceptable ones.

Purpose: To create justified confidence in correctness, robustness, safety, security, availability, reliability, understandability, modifiability, cost-effectiveness, and usability.

Rules and Steps

- Define what quality means for the approved project rather than using “high quality” as a slogan.
- Combine preventive and detective activities: requirements review, design review, prototype evaluation, code review, testing, profiling, and documentation review.
- Use QA for business/economic, legal, ethical, social, and professional reasons.
- Verification asks whether artifacts and implementation conform to specifications; validation asks whether the right system is being built for stakeholder needs.
- Quality evidence must be recorded and traceable.
- Quality is a lifecycle responsibility, not a final testing phase only.

Notation or Format

- Quality attributes and acceptance criteria; review records; test results; issue/defect status.
- The uploaded files do not define a formal QA plan template.

Examples in the Slides or Files

- Software QA slide asks about right system/right implementation, good/bad properties, errors, robustness, security, reliability, understandability and usability.
- Large-code challenge asks how to achieve maintainable, DRY, readable and bug-free code.
- Usability evaluation and code inspection are presented as QA mechanisms.

Mistakes to Avoid

- Equating QA only with test execution.
- No measurable quality criteria.
- Testing only at the end.
- Ignoring usability, maintainability, or security because the program runs.
- No evidence that reported defects were fixed and retested.
- No traceability from requirement to verification.

Effect on SUMS

SUMS must eventually define a QA strategy covering reviews, tests, usability, data integrity, security checks, and documentation consistency. Exact targets must come from approved requirements, not this knowledge base.

Source / Status	Sources: 09-codereviews.pdf, slides 2-8; 02-lifecycle.pdf, slide 6; 06-prototyping.pdf, slides 3-9. Status: Necessary for the System

20. Testing Concepts

Definition from the course materials: Testing executes or evaluates software behavior to reveal defects and provide evidence that functional and non-functional expectations are met. It is one part of QA.

Purpose: To verify components and integrations, validate use-case behavior, exercise failure paths, and reduce release risk.

Rules and Steps

- Use requirements as the basis for validation and verification.
- Use use-case main scenarios and extensions to derive success, error, and edge-case tests.
- Plan testing during requirements and design rather than after implementation.
- Distinguish levels mentioned in the slides: unit testing, function testing, and integration testing.
- Retest after fixes and protect previously working behavior with regression testing as a professional practice.
- Record expected result, actual result, status, evidence, and linked requirement/use case.
- Testing cannot prove the absence of all defects; combine it with inspections and reviews.

Notation or Format

- A formal test-case template is not supplied in the uploaded materials. A professional project format may include Test Case ID, requirement/use-case reference, preconditions, test data, steps, expected result, actual result, and pass/fail status.

Examples in the Slides or Files

- Slides report average defect-detection rates: unit testing 25%, function testing 35%, integration testing 45%, design inspection 55%, and code inspection 60%. These are lecture figures, not project acceptance thresholds.
- Use-case extensions are explicitly described as useful for discovering cases to test.
- The lifecycle includes testing, profiling, and QA.

Mistakes to Avoid

- Happy-path-only testing.
- No tests for validation, boundaries, failures, or integration.
- Inventing a coverage target not specified by the instructor.
- Confusing the lecture's defect-detection statistics with required project percentages.
- No link to requirements.
- Fixing a bug without retesting.
- Claiming "fully tested" when no test scope is documented.

Effect on SUMS

SUMS should later maintain test cases with stable TC identifiers and requirement traceability. The official files do not specify a required test-document format or coverage level, so these must be approved as project practices.

Source / Status	Sources: 09-codereviews.pdf, slides 3 and 7; 02-lifecycle.pdf, slide 6; 03-requirements.pdf, slide 4; 04-usecases.pdf, slides 3 and 11-14. Status: Necessary for the System + Professional Addition

21. Official Course Project Requirements

Definition from the course materials: The official requirements are the explicit instructions in final project.pdf and the selected SUMS description in Project list.pdf. They take precedence over illustrative examples and older lecture-course logistics.

Purpose: To establish the authoritative minimum obligations that the team must satisfy and that all project artifacts must support.

Rules and Steps

- Select one project; SUMS is the selected project and cannot be changed after assignment.
- Official SUMS description: a comprehensive platform to manage students, instructors, courses, registration, grading, attendance, and academic reports. This is a short scope statement, not a complete SRS.
- Team size is 3-4 students; all members contribute equally.
- Follow SDLC.
- Apply OOP throughout implementation.
- Use a relational database.
- Provide a clear, user-friendly interface.
- All work must be original; copying previous-semester, Internet, GitHub, or other-group projects is prohibited.
- Every member must be able to explain the part developed.
- GitHub is encouraged for versioning and collaboration.
- Submit the required files in one ZIP before the deadline of 30-07-2026.

Notation or Format

- The official files do not assign requirement IDs. This project will later assign stable IDs without changing the official wording.

Examples in the Slides or Files

- Project objectives: analyze requirements, apply software engineering, design professional UML, design/implement a relational database, develop a complete OOP application, work as a team, document professionally, and present/demonstrate the system.
- Hospital example demonstrates a possible workflow: requirement analysis; Use Case, Class, Sequence and ER diagrams; prototyping; implementation; documentation.

Mistakes to Avoid

- Treating the short SUMS description as a complete specification.
- Using the lecture's 5-6 team size instead of the official 3-4 rule.
- Copying example code or online projects.
- Submitting an application without a relational database or OOP structure.
- Unequal contribution or inability to explain work.
- Presenting an illustrative example requirement as an official SUMS requirement.

Effect on SUMS

All later SUMS baselines must map back to these official obligations. Any interpretation beyond the short scope statement must be elicited, recorded, and approved rather than silently assumed.

Source / Status	Sources: final project.pdf, pages 1-2; Project list.pdf, page 1; project example.pdf, pages 1-8 (illustrative only). Status: Officially Required

22. Required Deliverables

Definition from the course materials: Deliverables are the files and demonstrable outputs that must be submitted or are strongly expected to prove the required engineering process and product.

Purpose: To ensure the final package is complete, reviewable, runnable, explainable, and aligned with the official assignment.

Rules and Steps

- Official submissions: project source code; project report in PDF; SQL database file; presentation slides in PowerPoint or PDF; demonstration video if requested; GitHub repository link if available; all compressed into one ZIP.
- The report should provide sufficient professional documentation to explain analysis, design, database, implementation, testing/quality evidence, and use of SDLC.
- The project example strongly indicates requirement analysis, Use Case Diagram, Class Diagram, Sequence Diagram, ER Diagram, UI prototype, working MVP/application, and implementation documentation.
- Do not call an expected or professional artifact “officially required” unless the instructor confirms it.
- Ensure all final files use consistent names, versions, terminology, identifiers, and diagrams.
- Test the final ZIP, source build/run process, and SQL import on a clean environment before submission.

Notation or Format

- Recommended file manifest with status: Required, Conditional, Strongly Expected, or Professional Addition.
- The official assignment does not prescribe exact file names or report page count.

Examples in the Slides or Files

- Official list in final project.pdf.
- Hospital example workflow and diagrams.
- Lecture artifacts that may support the report: use-case specifications, prototypes, UML, review and test evidence.

Mistakes to Avoid

- Missing SQL file or source dependencies.
- Submitting only screenshots instead of source code.
- A PDF report that contradicts the code or diagrams.
- Broken repository links.
- Including temporary, secret, or generated files in the ZIP.
- Submitting optional items while a mandatory item is incomplete.
- No installation/run instructions when the evaluator cannot start the system.

Effect on SUMS

SUMS should maintain a live deliverables checklist from the beginning. Exact internal report contents and optional supporting files must be approved before final packaging.

Source / Status	Sources: final project.pdf, page 2; project example.pdf, pages 1-8. Status: Officially Required + Strongly Expected

A. Official Course Requirements Matrix

ID	Official Requirement	Source	Page	Output / Evidence	Responsible Phase
OCR-01	Select one project from the provided list.	final project.pdf	p.1	Approved project selection record	Initiation / Master Control
OCR-02	The selected project is Smart University Management System.	Project list.pdf	p.1	Project charter and scope source	Initiation
OCR-03	SUMS covers students, instructors, courses, registration, grading, attendance, and academic reports at description level.	Project list.pdf	p.1	Scope analysis; later SRS	Requirements
OCR-04	The assigned project cannot be changed.	final project.pdf	p.1	Decision baseline	Master Control
OCR-05	Team consists of 3-4 students.	final project.pdf	pp.1-2	Team record	Team Setup
OCR-06	All members contribute equally.	final project.pdf	p.2	Responsibility/contribution evidence	All phases
OCR-07	Analyze real-world software requirements.	final project.pdf	p.1	Requirements analysis / SRS content	Requirements
OCR-08	Apply Software Engineering principles.	final project.pdf	p.1	SDLC records and professional artifacts	All phases
OCR-09	Follow the Software Development Life Cycle.	final project.pdf	p.2	Phase plan and phase deliverables	Planning / All phases
OCR-10	Design professional UML diagrams.	final project.pdf	p.1	UML set in report/supporting files	System Design
OCR-11	Apply Object-Oriented Programming throughout implementation.	final project.pdf	pp.1-2	OOP source code and class design	Design / Implementation
OCR-12	Design and implement a relational database.	final project.pdf	pp.1-2	ERD/schema and SQL database file	Database Design / Implementation
OCR-13	Develop a complete software application.	final project.pdf	p.1	Runnable application source	Implementation / Release
OCR-14	Provide a clear, user-friendly interface.	final project.pdf	p.2	UI design and implemented screens	UI/UX / Implementation
OCR-15	Work effectively as a software development team.	final project.pdf	p.1	Responsibilities, coordination, contribution evidence	All phases
OCR-16	Prepare professional software documentation.	final project.pdf	p.1	Project Report PDF	Documentation
OCR-17	Present and demonstrate the developed system.	final project.pdf	p.1	Slides and live/demo evidence	Presentation
OCR-18	All submitted work must be original.	final project.pdf	p.2	Original code, diagrams, text, and assets	All phases / Audit
OCR-19	Copying from previous semesters, Internet, GitHub, or other groups is prohibited.	final project.pdf	p.2	Originality audit	All phases / Audit

ID	Official Requirement	Source	Page	Output / Evidence	Responsible Phase
OCR-20	Every member can explain the part developed.	final project.pdf	p.2	Team demo and Q&A readiness	All phases / Presentation
OCR-21	GitHub is encouraged for version management and collaboration.	final project.pdf	p.2	Repository (recommended)	Implementation / Collaboration
OCR-22	Submit Project Source Code.	final project.pdf	p.2	Source code folder/archive	Final Submission
OCR-23	Submit Project Report in PDF.	final project.pdf	p.2	Project_Report.pdf	Documentation / Submission
OCR-24	Submit SQL Database File.	final project.pdf	p.2	Database.sql	Database / Submission
OCR-25	Submit Presentation Slides in PowerPoint or PDF.	final project.pdf	p.2	Presentation.pptx or .pdf	Presentation / Submission
OCR-26	Submit Demonstration Video if requested.	final project.pdf	p.2	Demo video (conditional)	Demonstration / Submission
OCR-27	Provide GitHub Repository Link if available.	final project.pdf	p.2	Repository link (conditional)	Submission
OCR-28	Compress all files into one ZIP and upload to the course platform.	final project.pdf	p.2	Final submission ZIP	Final Submission
OCR-29	Deadline is 30-07-2026.	final project.pdf	p.2	Submitted package before deadline	Schedule / Final Submission

B. Required Deliverables Checklist

ID	Deliverable	Authority	Acceptance Check	Current Status
B-01	Project Source Code	Official - Mandatory	Complete source tree; no missing dependencies; clean build/run check	Not started
B-02	Project Report	Official - Mandatory	PDF; professional and internally consistent	Not started
B-03	SQL Database File	Official - Mandatory	Schema and required SQL; clean import check	Not started
B-04	Presentation Slides	Official - Mandatory	PowerPoint or PDF	Not started
B-05	Final ZIP Package	Official - Mandatory	Contains all required files; opens successfully	Not started
B-06	Project Demonstration Video	Conditional - if requested	Playable file/link with agreed duration	Awaiting instructor clarification
B-07	GitHub Repository Link	Conditional - if available / encouraged	Accessible repository and suitable history	Recommended
B-08	Requirement Analysis	Strongly Expected by example	Report section and/or approved SRS	Not started
B-09	Use Case Diagram	Strongly Expected by example + UML objective	Professional UML diagram	Not started
B-10	Class Diagram	Strongly Expected by example + OOP objective	Professional UML class model	Not started
B-11	Sequence Diagram(s)	Strongly Expected by example	Approved scenario diagrams	Not started
B-12	ER Diagram / Relational Design	Strongly Expected by example + DB objective	ERD/schema in report	Not started
B-13	UI Prototype / Wireframes	Strongly Expected by example	Prototype evidence linked to use cases	Not started
B-14	Working Application / MVP	Official complete application; example says MVP	Runnable release meeting approved scope	Not started
B-15	Implementation Description	Strongly Expected by example	Report section describing architecture/code/database	Not started
B-16	SRS Document	Professional Addition	Separate or embedded approved requirements baseline	Planned; not official
B-17	Formal Use Case Specifications	Professional Addition	Critical use cases with extensions	Planned; count TBD
B-18	Architecture Description	Professional Addition / Necessary	Layer/module decisions and rationale	Planned; not official
B-19	Test Plan and Test Cases	Professional Addition / Necessary	Traceable test evidence	Planned; not official
B-20	README and Installation Guide	Professional Addition / Necessary for	Clean setup and run instructions	Planned; not official

ID	Deliverable	Authority	Acceptance Check	Current Status
		evaluation		
B-21	Decisions, Assumptions, Risks, Changes Logs	Professional Addition	Maintained in Master Project Control	Active governance
B-22	Contribution Record	Necessary for official equal-contribution rule	Artifact/code ownership and review evidence	Planned

C. UML Rules Matrix

Diagram	Element	Rule	Notation	Common Error
Use Case Diagram	Actor	External behavioral role; primary initiates, supporting assists.	Stick figure.	Do not use database/internal class as actor unless genuinely external.
Use Case Diagram	Use Case	Actor goal / observable interaction.	Oval with goal-oriented verb phrase.	Avoid internal operations or vague “manage everything.”
Use Case Diagram	Association	Actor participates in use case.	Solid line in course examples.	Do not connect all actors to all use cases.
Use Case Diagram	Advanced relations	Not formally taught in uploaded slides.	<<include>>, <<extend>>, generalization only under approved convention.	Do not overuse or claim they are explicit slide requirements.
Class Diagram	Class	One key abstraction.	Three compartments: name, attributes, operations.	Avoid God class and proliferation of tiny classes.
Class Diagram	Attribute	Field/property with visibility and type.	visibility name : type [count] = defaultValue.	Do not omit types or expose all fields publicly.
Class Diagram	Operation	Public/protected/private behavior.	visibility name(parameters) : returnType.	Do not repeat inherited methods; may omit trivial get/set.
Class Diagram	Visibility/static/derived	Access and storage semantics.	+ # - ~; underline static; / derived.	Use symbols consistently.
Class Diagram	Inheritance	Child is a specialized parent.	Arrow points upward to parent; slide line styles vary by parent type.	Do not model mere roles as subclasses.
Class Diagram	Association/multiplicity	Structural relationship and quantity.	1, *, 2..4, 3..*; name; navigability.	Do not omit multiplicity on important relationships.
Class Diagram	Aggregation/composition/dependency	Part-of, life-cycle ownership, or temporary use.	White diamond; black diamond; dotted line/arrow.	Do not use composition unless parts live/die with whole.
Sequence Diagram	Participant/lifeline	Object/entity in one scenario.	objectName : ClassName over dashed vertical lifeline.	Avoid generic participants with no clear responsibility.
Sequence Diagram	Message/return	Communication over time.	Call arrow with name/arguments; dashed return.	Order must match scenario and class operations.
Sequence Diagram	Activation	Method active/on stack.	Narrow activation bar; nesting for recursion.	Do not omit all activations in a detailed diagram.
Sequence Diagram	Frames	Control alternatives and repetition.	opt [condition], alt [condition], loop [condition/items], ref.	Always state guards/conditions clearly.
Sequence Diagram	Creation/destruction	Lifetime begins/ends during scenario.	new/create arrow; X at lifeline end.	Use only when relevant.
Flowchart	Terminal	Start/end.	Rounded rectangle.	Every valid path needs a defined ending.
Flowchart	Process	Calculation or assignment.	Rectangle.	Do not use it for reading/printing.
Flowchart	Input/Output	Read/display data.	Parallelogram.	Label data clearly.
Flowchart	Decision	Binary condition.	Diamond with Yes/No or True/False branches.	No unlabeled branches.

Diagram	Element	Rule	Notation	Common Error
Flowchart	Connector/predefined process/flow	Avoid crossings, call subroutine, show direction.	Circle; double-sided rectangle; arrows.	Avoid loops without update/termination.

D. Project Workflow

Phase 0 - Master Project Control Setup

Create authoritative logs, versioning rules, source hierarchy, team responsibilities, schedule, and approval process. Output: project-control baseline.

Phase 1 - Official Scope and Requirements Elicitation

Analyze the official SUMS description, identify stakeholders, questions, constraints, assumptions, risks, and system boundary. Do not create final requirements before resolving ambiguity.

Phase 2 - SRS and Requirements Baseline

Create and review FR, NFR, business rules, security constraints, glossary, scope, priorities, and traceability. Approve the baseline before design.

Phase 3 - Actors and Use Cases

Create actor/goal list, top-level Use Case Diagram, and selected formal use-case specifications with extensions. Verify coverage against requirements.

Phase 4 - System and Object Design

Approve architecture/layers/modules, class responsibilities, Class Diagram, and critical Sequence Diagrams. Record design decisions.

Phase 5 - Database Design

Create conceptual ERD, relational schema, keys, constraints, normalization decisions, data dictionary, and SQL plan. Verify support for approved requirements.

Phase 6 - UI/UX and Prototyping

Create task flows, wireframes/prototypes, conduct walkthroughs/usability checks, and approve screens. Each screen must support an approved use case.

Phase 7 - Implementation Planning

Choose technology stack, repository workflow, coding standards, module boundaries, milestones, definition of done, and test approach. No unapproved technology constraints should appear earlier as requirements.

Phase 8 - Incremental Implementation and Integration

Implement approved scope in small coherent changes. Use OOP, relational database, version control, reviews, frequent integration, and progress evidence.

Phase 9 - Verification, Validation and QA

Execute unit/function/integration/system tests, usability checks, data-integrity checks, security checks, defect tracking, fixes, regression tests, and traceability review.

Phase 10 - Documentation and Presentation

Complete report, README/install guide, diagrams, SQL, slides, demo script/video if requested, contribution evidence, and team rehearsal.

Phase 11 - Final Audit and Submission

Clean-environment run, SQL import, link check, originality check, consistency audit, file manifest, ZIP validation, approval, and submission before 30-07-2026.

Phase Gate Rule

> Implementation must not begin as the project's main activity before requirements, scope, SRS, core UML, architecture, database design, and UI/UX decisions have been reviewed and approved. Small disposable technical experiments may be approved separately, but they must not silently become the production baseline.

E. Missing and Ambiguous Information

- No grading rubric or mark distribution is included.
- No official project-report template, page count, section order, font, or citation style is specified.
- No exact number of functional/non-functional requirements is specified.
- No official list of SUMS actors, permissions, business rules, or workflows is provided.
- No required number of use cases, formal specifications, class diagrams, sequence diagrams, or ER diagrams is specified.
- No explicit list of UML diagram types appears in the official submission list; the example strongly suggests Use Case, Class, Sequence, and ER diagrams.
- No database management system is mandated.
- No programming language, framework, architecture style, or target platform (web/desktop/mobile) is mandated.
- No definition of “complete software application” or minimum accepted feature set is given beyond the short SUMS description.
- The example says “working MVP,” while the official objective says “complete software application”; the expected completeness level needs clarification.
- No performance, availability, security, privacy, concurrent-user, or data-volume targets are specified.
- No authentication/authorization requirements are stated.
- No exact UI screen count, fidelity, responsiveness, accessibility, or browser/device support is specified.
- No required test types, coverage percentage, test-case count, or test-report template is specified.
- No code-review record is required even though code review is taught.
- No architecture, component, deployment, activity, state, or data-flow diagram is explicitly required.
- No formal SRS file is explicitly named as a submission.
- No README, installation guide, user manual, deployment guide, or data dictionary is explicitly required.
- No demonstration format, duration, live-demo rule, or video deadline is specified.
- The demonstration video is conditional (“Optional if requested”), but the request trigger is not defined.
- GitHub is encouraged and the repository link is “if available”; whether repository history affects grading is unknown.
- The deadline states 30-07-2026 but gives no exact time or timezone.
- No ZIP naming convention or internal folder structure is specified.
- No presentation duration, number of presenters, slide limit, or Q&A format is specified.
- No policy is given for third-party libraries, templates, icons, generated assets, or open-source licenses, beyond the originality/no-copying rule.
- No instructor approval process is specified for assumptions or scope changes.
- Lecture 1 mentions 5-6 member teams, but final project.pdf requires 3-4; the official assignment resolves this conflict in favor of 3-4.
- The Hospital example’s UML is simplified and contains formatting/notation weaknesses; whether the instructor expects this simplified level or formal lecture notation is not explicitly stated.
- No separate lecture on relational database design or full testing methodology is present in the uploaded source set, despite both being important to the project.

F. Common Mistakes to Avoid

- Starting code before approved analysis and design.
- Treating the short SUMS description as a complete SRS.
- Inventing actors, rules, permissions, grade policies, or performance targets without approval.
- Adding optional “smart” features before completing official core scope.
- Using different terminology or identifiers across SRS, diagrams, database, UI, code, and tests.
- Submitting UML that does not follow the notation taught in the slides.
- Copying the Hospital example’s simplified diagrams without correcting notation and adapting from approved SUMS requirements.
- Confusing a Class Diagram with an ER Diagram.
- Building a God class or one generic Manager/System class for most logic.
- Missing multiplicities, inconsistent method names, or sequence messages that are absent from classes.
- Creating screens that have no approved use case.
- Using a database as a storage afterthought rather than designing constraints and relationships.
- Using OOP syntax without real encapsulation, responsibility distribution, or polymorphic design where justified.
- No version control or infrequent massive commits.
- No code review or integration until the final week.
- Only happy-path testing; no validation, boundary, error, or integration tests.
- No clean-environment run or SQL import test before submission.

- Report, code, and slides describe different versions of the system.
- Unequal contribution or a member unable to explain the project.
- Copied text/code/assets or a repository that resembles an existing project.
- Missing mandatory file inside the final ZIP.
- Broken links, absolute local paths, passwords/secrets, or environment-specific configuration.
- Submitting after the deadline because packaging and upload were left to the last minute.
- Calling the knowledge base or any later artifact “approved” before Master Project Control records approval.

G. Recommendations for SUMS

- Use this approved document as the authoritative course-reference baseline for all subsequent SUMS phases.
- Keep the official SUMS description unchanged as the scope source, then use elicitation and approved assumptions to refine it later.
- Use stable identifiers and a traceability matrix from the first requirement baseline.
- Separate official requirements, necessary system decisions, professional additions, and future enhancements in every phase.
- Use a staged, iterative workflow: stable upstream baselines plus early prototypes and incremental implementation.
- Keep the first implementation scope realistic for 3-4 students and the deadline.
- Create one authoritative glossary and reuse names everywhere.
- Derive use cases from actors/goals, classes from requirements/use cases, database entities from approved information needs, screens from use cases, and tests from requirements and extensions.
- Select technology only after requirements and architecture are sufficiently understood; record the decision and rationale.
- Use a layered architecture and separation of UI, business logic, and data access as a likely professional direction, but do not approve a specific architecture in this knowledge-base phase.
- Use GitHub or equivalent version control with small commits, branch/review practice, issue tracking, and protected final baseline.
- Plan QA, security, validation, error handling, and documentation from the beginning.
- Create a clean installation/run procedure so the evaluator can operate the system.
- Schedule early internal demonstrations and a final rehearsal where every member explains analysis, diagrams, database, code, and testing.
- Ask the instructor for the missing rubric, exact diagram expectations, technology constraints, testing expectations, presentation format, and final upload details before locking the SRS.

Appendix 1. Source Coverage Map

Source	Coverage	Pages/Slides
01-introduction.pdf	Software engineering definition, roles, project trade-offs, failure causes, mistake categories, teamwork/source-control advice.	Slides 1-13
02-lifecycle.pdf	SDLC phases; code-and-fix, Waterfall, Spiral, evolutionary prototyping, staged/evolutionary delivery, design-to-schedule, Agile; model evaluation.	Slides 1-21
03-requirements.pdf	Requirements definition, policy/mechanism, elicitation, feature creep, DRY, good/bad requirements, FR/NFR, Cockburn template.	Slides 1-14
04-usecases.pdf	Actors/stakeholders/goals, use-case styles, informal/formal use cases, extensions, Cockburn steps, user stories.	Slides 1-16
05-teams.pdf	Roles, team structures, decisions, responsibilities, productivity, contribution problems, meetings.	Slides 1-19
06-prototyping.pdf	Usability, prototyping methods/timing, paper sessions, Golden Rules, UI components, paper prototype construction.	Slides 1-28
07-classdiagrams.pdf	UML/class syntax, visibility, methods, inheritance, associations/multiplicity, aggregation/composition/dependency, design heuristics.	Slides 1-29
08-sequencediagrams.pdf	Participants, lifelines, messages, returns, lifetime, activations, frames, references, examples and flaws.	Slides 1-22
09-codereviews.pdf	QA questions, code-review definition/mechanics/value, inspection/walkthrough/reading, industry examples, checklist, code refactoring example.	Slides 1-21
Flowchart Exercises 1 File.pdf	Flowchart definition/symbols and basic algorithm exercises.	Pages 1-5
Flowchart Exercises 2 File.pdf	Worked flowchart examples with sequence, decisions, loops, trace table, grading and aggregation exercises.	Pages 1-18
final project.pdf	Official objectives, team rules, SDLC/OOP/database/UI/originality rules, submissions, deadline.	Pages 1-2
Project list.pdf	Official selected SUMS title and short scope statement.	Page 1
project example.pdf	Illustrative Hospital workflow: Use Case, Class, Sequence, ER, prototyping, implementation/MVP, documentation, code snippets.	Pages 1-8

Appendix 2. Baseline Preservation Rules

- Every approved functional requirement retains its ID permanently.
- Every approved non-functional requirement, business rule, security requirement, use case, test case, decision, assumption, and risk retains its ID.
- A change does not overwrite history; it is recorded in the Decisions/Change Log with reason, impact, owner, date, and approval.
- All downstream artifacts must be rechecked when an upstream item changes.
- No database entity, class, screen, API, or test is approved without traceability to an approved need or design decision.
- Artifacts use version numbers and status values such as Draft, Review, Approved, Superseded, and Rejected.
- The Master Project Control chat is authoritative for approvals, baselines, and final versions.
- This v1.0 knowledge base is the approved course-reference baseline. Any modification requires a documented change decision.

Review Decision

Decision Option	Selection / Signature
Approve v1.0 as Course Knowledge Base baseline	

Decision Option	Selection / Signature
Approve with changes listed below	
Return for major revision	

Required changes / notes:

_____

_____

_____

_____

_____